=== FILE: map_editor/config.py ===

```
"""Конфигурация редактора карт — полностью автоматическая"""

import os
import pygame
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG

# ============================================================
# РАЗМЕРЫ
# ============================================================
WINDOW_WIDTH = 1200
WINDOW_HEIGHT = 800
DEFAULT_CELL_SIZE = 20
MIN_CELL_SIZE = 6
MAX_CELL_SIZE = 80

# ============================================================
# БАЗОВЫЕ ЦВЕТА (только для фона и интерфейса)
# ============================================================
COLORS = {
    'background': (30, 30, 35),
    'text': (240, 240, 240),
    'text_dim': (160, 160, 170),
    'grid': (80, 80, 90),
    'selection': (255, 255, 255, 128),
    'panel_bg': (45, 45, 50),
    'panel_border': (80, 80, 90),
    'info_bg': (35, 35, 40),
}

# ============================================================
# АВТОМАТИЧЕСКИЕ ЦВЕТА (fallback, если нет текстуры)
# ============================================================
def generate_symbol_colors():
    """Генерирует цвета для всех объектов из конфигов"""
    colors = {}

    # 1. Стены — серые
    for symbol, config in SYMBOLS_CONFIG.items():
        if config.get('type') == 'wall':
            colors[symbol] = (60, 60, 70)

    # 2. Пол — тёмный
    colors['_'] = (20, 20, 25)

    # 3. Спавн — жёлтый
    colors['S'] = (100, 80, 0)

    # 4. Выход — зелёный
    colors['E'] = (0, 100, 0)

    # 5. Двери — серые
    for symbol, config in SYMBOLS_CONFIG.items():
        if config.get('type') == 'door':
            door_type = config.get('door_type', 'normal')
            if door_type == 'secret':
```

```
                colors[symbol] = (100, 80, 60)
            elif door_type == 'key_red':
                colors[symbol] = (200, 50, 50)
            elif door_type == 'key_blue':
                colors[symbol] = (50, 100, 200)
            elif door_type == 'key_yellow':
                colors[symbol] = (200, 200, 50)
            else:
                colors[symbol] = (100, 100, 100)

    # 6. Предметы
    item_colors = {
        'health': (200, 0, 0),
        'armor': (0, 100, 200),
        'key_red': (200, 0, 0),
        'key_blue': (0, 100, 200),
        'key_yellow': (200, 200, 0),
    }

    for symbol, config in SYMBOLS_CONFIG.items():
        if config.get('type') == 'item':
            item_type = config.get('item_type')
            if item_type in item_colors:
                colors[symbol] = item_colors[item_type]
            elif config.get('weapon_name'):
                colors[symbol] = (200, 200, 0)  # Оружие — жёлтое
            else:
                colors[symbol] = (150, 150, 150)

    # 7. NPC
    npc_colors = {
        '2': (200, 50, 50),
        '3': (200, 100, 0),
        '4': (150, 50, 200),
        '5': (50, 200, 200),
        '6': (200, 0, 200),
    }

    for symbol in NPC_CONFIG.keys():
        colors[symbol] = npc_colors.get(symbol, (200, 100, 100))

    return colors

SYMBOL_COLORS = generate_symbol_colors()

# ============================================================
# ПУТИ
# ============================================================
ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RESOURCES_DIR = os.path.join(ROOT_DIR, 'resources')
TEXTURES_DIR = os.path.join(RESOURCES_DIR, 'textures')
NPC_DIR = os.path.join(RESOURCES_DIR, 'npc')
LEVELS_DIR = os.path.join(RESOURCES_DIR, 'levels')
BACKUP_DIR = os.path.join(LEVELS_DIR, 'levels_backup')
GAME_DATA_PATH = os.path.join(ROOT_DIR, 'config', 'game_data.py')
```

=== FILE: map_editor/config_loader.py ===

```
# map_editor/config_loader.py
import sys
import os

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT_DIR not in sys.path:
    sys.path.insert(0, ROOT_DIR)

from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG, WEAPON_CONFIG
```

=== FILE: map_editor/editor.py ===

```
"""Главный класс редактора"""

import os
import json
import pygame
from .config import *
from .ui import Canvas, InfoPanel
from .ui.toolbar import Toolbar
from .ui.tools_panel import ToolsPanel
from .tools import Brush, Eraser
from .tools.selection import Selection


class MapEditor:
    """Редактор карт"""

    def __init__(self, level_file=None):
        pygame.init()
        pygame.display.set_caption("Map Editor — Загрузите уровень")

        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        self.clock = pygame.time.Clock()
        self.running = True

        self.dialog_active = False
        self.dialog_choice = None

        self.grid = []
        self.current_file = None
        self.selected_symbol = 'M'
        self.has_changes = False
        self._saving = False

        self._setup_ui()
        self._setup_tools()

        if level_file:
            self.load_level(level_file)

    def _setup_ui(self):
        # ПАНЕЛЬ ИНСТРУМЕНТОВ (сверху) - ВЫСОТА 50px
        tools_rect = pygame.Rect(0, 0, WINDOW_WIDTH - 250, 50)
        self.tools_panel = ToolsPanel(tools_rect)

        # КАНВАС (под панелью инструментов)
        canvas_rect = pygame.Rect(0, 50, WINDOW_WIDTH - 250, WINDOW_HEIGHT - 50 - 60)
        self.canvas = Canvas(canvas_rect)

        # ИНФОРМАЦИЯ (внизу)
        info_rect = pygame.Rect(0, WINDOW_HEIGHT - 60, WINDOW_WIDTH - 250, 60)
        self.info_panel = InfoPanel(info_rect)

        # ЛЕГЕНДА (справа)
        toolbar_rect = pygame.Rect(WINDOW_WIDTH - 250, 0, 250, WINDOW_HEIGHT)
        self.toolbar = Toolbar(toolbar_rect)
```

```
def _setup_tools(self):
    self.brush = Brush(self)
    self.eraser = Eraser(self)
    self.selection = Selection(self)
    self.current_tool = 'brush'

def _on_change(self):
    if not self._saving:
        self.has_changes = True
        self._auto_save()

def _auto_save(self):
    if self._saving or not self.current_file or not self.grid or not self.has_changes:
        return

    self._saving = True
    self._create_backup()
    self._saving = False

def _create_backup(self):
    if not self.current_file or not self.grid:
        return

    backup_dir = os.path.join(os.path.dirname(self.current_file), 'levels_backup')
    os.makedirs(backup_dir, exist_ok=True)

    base_name = os.path.basename(self.current_file)
    name, ext = os.path.splitext(base_name)
    backup_path = os.path.join(backup_dir, f"{name}_backup{ext}")

    try:
        with open(self.current_file, 'r', encoding='utf-8') as f:
            data = json.load(f)

        data['map'] = self.grid

        with open(backup_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)

    except Exception as e:
        print(f"[Бэкап] Ошибка: {e}")

def _clear_backups(self):
    if not self.current_file:
        return

    backup_dir = os.path.join(os.path.dirname(self.current_file), 'levels_backup')
    if not os.path.exists(backup_dir):
        return

    base_name = os.path.basename(self.current_file)
    name, _ = os.path.splitext(base_name)

    for f in os.listdir(backup_dir):
        if f.startswith(name) and f.endswith('_backup.json'):
            try:
                os.remove(os.path.join(backup_dir, f))
            except:
```

```
                pass

    def load_level(self, file_path):
        if not os.path.exists(file_path):
            print(f"[Ошибка] Файл не найден: {file_path}")
            return False

        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

            map_data = data.get('map', [])

            # Если map_data — список строк, превращаем в список списков
            if map_data and isinstance(map_data[0], str):
                self.grid = [list(row) for row in map_data]
            else:
                self.grid = map_data

            self.current_file = file_path
            self.has_changes = False

            self.canvas.set_grid(self.grid)

            filename = os.path.basename(file_path)
            pygame.display.set_caption(f"Map Editor — {filename}")

            print(f"[Успех] Загружен уровень: {file_path}")
            print(f"  Размер: {len(self.grid[0])}x{len(self.grid)}")

            return True

        except Exception as e:
            print(f"[Ошибка] Не удалось загрузить уровень: {e}")
            return False

    def save_level(self, file_path=None):
        if file_path is None:
            file_path = self.current_file

        if not file_path or not self.grid:
            return False

        try:
            # Загружаем существующие данные
            data = {}
            if os.path.exists(file_path):
                with open(file_path, 'r', encoding='utf-8') as f:
                    data = json.load(f)

            # Обновляем карту
            data['map'] = self.grid

            # ============================================================
            # РУЧНОЕ ФОРМАТИРОВАНИЕ
            # ============================================================
            # Форматируем map вручную
            map_lines = []
            for row in self.grid:
```

```
            json_row = json.dumps(row, ensure_ascii=False)
            map_lines.append(f"    {json_row}")
        map_json = "[\n" + ",\n".join(map_lines) + "\n  ]"

        # Форматируем остальные данные
        meta_data = {k: v for k, v in data.items() if k != 'map'}
        meta_json = json.dumps(meta_data, indent=4, ensure_ascii=False)

        # Собираем финальный JSON
        final_json = "{\n" + f'  "map": {map_json},\n' + meta_json[2:]

        with open(file_path, 'w', encoding='utf-8') as f:
            f.write(final_json)

        self.has_changes = False
        self._clear_backups()

        print(f"[Сохранено] {file_path}")
        return True

    except Exception as e:
        print(f"[Ошибка] Сохранение: {e}")
        return False

def _open_dialog(self):
    if self.has_changes and self.current_file:
        self.dialog_active = True
        self.dialog_choice = None
    else:
        self.running = False

def _draw_dialog(self):
    overlay = pygame.Surface((WINDOW_WIDTH, WINDOW_HEIGHT))
    overlay.set_alpha(180)
    overlay.fill((0, 0, 0))
    self.screen.blit(overlay, (0, 0))

    dialog_rect = pygame.Rect(
        WINDOW_WIDTH // 2 - 220,
        WINDOW_HEIGHT // 2 - 80,
        440, 160
    )
    pygame.draw.rect(self.screen, (50, 50, 60), dialog_rect)
    pygame.draw.rect(self.screen, (100, 100, 120), dialog_rect, 2)

    font = pygame.font.Font(None, 24)
    font_small = pygame.font.Font(None, 18)

    title = font.render("Сохранить изменения?", True, (255, 255, 255))
    title_rect = title.get_rect(center=(dialog_rect.centerx, dialog_rect.y + 30))
    self.screen.blit(title, title_rect)

    y = dialog_rect.y + 70
    options = [
        ("[Y] Да, сохранить и выйти", (dialog_rect.x + 30, y)),
        ("[N] Нет, выйти без сохранения", (dialog_rect.x + 30, y + 25)),
        ("[ESC] Отмена", (dialog_rect.x + 30, y + 50)),
    ]
```

```
    for text, pos in options:
        rendered = font_small.render(text, True, (200, 200, 200))
        self.screen.blit(rendered, pos)

def _handle_events(self):
    for event in pygame.event.get():
        if self.dialog_active:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_y:
                    self.dialog_choice = 'save'
                    self.dialog_active = False
                elif event.key == pygame.K_n:
                    self.dialog_choice = 'discard'
                    self.dialog_active = False
                elif event.key == pygame.K_ESCAPE:
                    self.dialog_choice = 'cancel'
                    self.dialog_active = False
            continue

        if event.type == pygame.QUIT:
            self._open_dialog()
            continue

        # ПАНЕЛЬ ИНСТРУМЕНТОВ
        if event.type == pygame.MOUSEBUTTONDOWN:
            mx, my = event.pos
            if self.tools_panel.handle_click(mx, my):
                self.current_tool = self.tools_panel.get_selected_tool()
                if self.current_tool != 'select':
                    self.selection.start_x = None
                    self.selection.start_y = None
                    self.selection.end_x = None
                    self.selection.end_y = None
                continue

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                self._open_dialog()
                continue

            if event.key == pygame.K_s and (event.mod & pygame.KMOD_CTRL):
                if self.current_file:
                    self.save_level(self.current_file)
                continue

            if event.key == pygame.K_0 and (event.mod & pygame.KMOD_CTRL):
                self.canvas._center_view()

            if event.key == pygame.K_DELETE:
                if self.current_tool == 'select' and self.selection.get_selection():
                    self.selection.clear()

        elif event.type == pygame.MOUSEBUTTONDOWN:
            mx, my = event.pos

            if self.toolbar.rect.collidepoint(mx, my):
                if event.button == 4:
                    self.toolbar.scroll(-30)
                    continue
```

```
            elif event.button == 5:
                self.toolbar.scroll(30)
                continue

        if self.toolbar.handle_click(mx, my):
            symbol = self.toolbar.get_selected_symbol()
            self.selected_symbol = symbol
            if self.current_tool == 'select' and self.selection.get_selection():
                self.selection.fill(symbol)
            continue

        if self.canvas.rect.collidepoint(mx, my):
            if event.button == 4:
                self.canvas.zoom_in()
                continue
            elif event.button == 5:
                self.canvas.zoom_out()
                continue

        if event.button == 2:
            self.canvas.start_drag(mx, my)
            continue

        cell = self.canvas.get_cell_at(mx, my)
        if not cell:
            continue

        x, y = cell

        if self.current_tool == 'brush' and event.button == 1:
            self.brush.apply(x, y)
        elif self.current_tool == 'eraser' and event.button == 1:
            self.eraser.apply(x, y)
        elif self.current_tool == 'select' and event.button == 1:
            self.selection.start(x, y)

    elif event.type == pygame.MOUSEBUTTONUP:
        if event.button == 2:
            self.canvas.end_drag()

        if self.current_tool == 'select' and event.button == 1:
            self.selection.end()

    elif event.type == pygame.MOUSEMOTION:
        mx, my = event.pos
        self.canvas.update_drag(mx, my)

        cell = self.canvas.get_cell_at(mx, my)

        if pygame.mouse.get_pressed()[0] and cell:
            x, y = cell
            if self.current_tool == 'brush':
                self.brush.apply(x, y)
            elif self.current_tool == 'eraser':
                self.eraser.apply(x, y)
            elif self.current_tool == 'select':
                self.selection.update(x, y)

        if pygame.mouse.get_pressed()[2] and cell:
```

```
                x, y = cell
                self.eraser.apply(x, y)

            symbol = None
            if cell:
                x, y = cell
                if 0 <= y < len(self.grid) and 0 <= x < len(self.grid[0]):
                    symbol = self.grid[y][x]
            self.info_panel.update(cell, symbol)
            self.canvas.update_hover(mx, my)

def _draw(self):
    self.screen.fill(COLORS['background'])

    self.canvas.draw(self.screen)

    if self.current_tool == 'select':
        self.selection.draw(self.screen)

    # Обновляем статус в info_panel
    self.info_panel.update_status(
        self.current_tool,
        self.selected_symbol,
        self.has_changes
    )

    self.info_panel.draw(self.screen)
    self.toolbar.draw(self.screen)
    self.tools_panel.draw(self.screen)

    if self.dialog_active:
        self._draw_dialog()

    pygame.display.flip()

def run(self):
    while self.running:
        self._handle_events()

        if not self.dialog_active and self.dialog_choice is not None:
            if self.dialog_choice == 'save':
                if self.current_file:
                    self.save_level(self.current_file)
                self.running = False
            elif self.dialog_choice == 'discard':
                self.running = False
            elif self.dialog_choice == 'cancel':
                self.dialog_choice = None

        self._draw()
        self.clock.tick(60)

    pygame.quit()
```

=== FILE: map_editor/history.py ===

=== FILE: map_editor/main.py ===

```
"""Точка входа в редактор карт"""

import sys
import os

# Добавляем корневую папку в путь
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from map_editor.editor import MapEditor


def main():
    """Запуск редактора"""
    level_file = None

    # Проверяем аргументы командной строки
    if len(sys.argv) > 1:
        level_file = sys.argv[1]

    editor = MapEditor(level_file)
    editor.run()


if __name__ == "__main__":
    main()
```

=== FILE: map_editor/validators.py ===

=== FILE: map_editor/tools/brush.py ===

```
"""Кисть — ставит выбранный объект в клетку"""

class Brush:
    """Инструмент 'Кисть'"""

    def __init__(self, editor):
        self.editor = editor

    def apply(self, x, y):
        """Ставит выбранный объект в клетку (x, y)"""
        if not self.editor.grid:
            return

        if 0 <= y < len(self.editor.grid) and 0 <= x < len(self.editor.grid[0]):
            symbol = self.editor.selected_symbol
            if self.editor.grid[y][x] != symbol:
                self.editor.grid[y][x] = symbol
                self.editor._on_change()
                return True
        return False
```

=== FILE: map_editor/tools/eraser.py ===

```
"""Ластик — ставит '_' в клетку"""

class Eraser:
    """Инструмент 'Ластик'"""

    def __init__(self, editor):
        self.editor = editor

    def apply(self, x, y):
        """Стирает клетку (ставит '_')"""
        if not self.editor.grid:
            return

        if 0 <= y < len(self.editor.grid) and 0 <= x < len(self.editor.grid[0]):
            if self.editor.grid[y][x] != '_':
                self.editor.grid[y][x] = '_'
                self.editor._on_change()
                return True
        return False
```

=== FILE: map_editor/tools/fill.py ===

=== FILE: map_editor/tools/selection.py ===

```
"""Инструмент 'Выделение'"""

import pygame


class Selection:
    """Инструмент для выделения области"""

    def __init__(self, editor):
        self.editor = editor
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.is_selecting = False

    def start(self, x, y):
        """Начинает выделение"""
        self.start_x = x
        self.start_y = y
        self.end_x = x
        self.end_y = y
        self.is_selecting = True

    def update(self, x, y):
        """Обновляет выделение (при движении мыши)"""
        if self.is_selecting:
            self.end_x = x
            self.end_y = y

    def end(self):
        """Заканчивает выделение"""
        self.is_selecting = False

    def get_selection(self):
        """Возвращает выделенную область (x1, y1, x2, y2)"""
        if self.start_x is None or self.end_x is None:
            return None

        x1 = min(self.start_x, self.end_x)
        y1 = min(self.start_y, self.end_y)
        x2 = max(self.start_x, self.end_x)
        y2 = max(self.start_y, self.end_y)

        return (x1, y1, x2, y2)

    def get_cells(self):
        """Возвращает список клеток в выделенной области"""
        sel = self.get_selection()
        if not sel:
            return []

        x1, y1, x2, y2 = sel
        cells = []
        for y in range(y1, y2 + 1):
```

```
            for x in range(x1, x2 + 1):
                if 0 <= y < len(self.editor.grid) and 0 <= x < len(self.editor.grid[0]):
                    cells.append((x, y))
        return cells

    def fill(self, symbol):
        """Заполняет выделенную область символом"""
        cells = self.get_cells()
        if not cells:
            return

        changed = False
        for x, y in cells:
            if self.editor.grid[y][x] != symbol:
                self.editor.grid[y][x] = symbol
                changed = True

        if changed:
            self.editor._on_change()

        # Сбрасываем выделение
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None

    def clear(self):
        """Очищает выделенную область (ставит '_')"""
        self.fill('_')
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None

    def draw(self, screen):
        """Рисует рамку выделения (вызывается из canvas)"""
        sel = self.get_selection()
        if not sel:
            return

        x1, y1, x2, y2 = sel
        # Преобразуем координаты клеток в пиксели
        rect_x = self.editor.canvas.rect.x + self.editor.canvas.scroll_x + x1 * self.editor.canvas.cell_si
ze
        rect_y = self.editor.canvas.rect.y + self.editor.canvas.scroll_y + y1 * self.editor.canvas.cell_si
ze
        rect_w = (x2 - x1 + 1) * self.editor.canvas.cell_size
        rect_h = (y2 - y1 + 1) * self.editor.canvas.cell_size

        # Рамка
        pygame.draw.rect(screen, (255, 255, 255), (rect_x, rect_y, rect_w, rect_h), 2)

        # Полупрозрачная заливка
        overlay = pygame.Surface((rect_w, rect_h))
        overlay.set_alpha(30)
        overlay.fill((255, 255, 255))
        screen.blit(overlay, (rect_x, rect_y))
```

=== FILE: map_editor/tools/__init__.py ===

```
"""Инструменты редактора"""

from .brush import Brush
from .eraser import Eraser
```

=== FILE: map_editor/ui/canvas.py ===

```
"""Отрисовка карты с текстурами (автоматический скейлинг + кэш)"""

import pygame
import os
from ..config import COLORS, SYMBOL_COLORS
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG


class Canvas:
    """Область отрисовки карты"""

    def __init__(self, rect):
        self.rect = rect
        self.grid = []
        self.width = 0
        self.height = 0
        self.cell_size = 20
        self.scroll_x = 0
        self.scroll_y = 0

        self.dragging = False
        self.drag_start_x = 0
        self.drag_start_y = 0
        self.scroll_start_x = 0
        self.scroll_start_y = 0

        self.hover_cell = None

        # Кэш текстур и шрифтов
        self.texture_cache = {}
        self.font_cache = {}
        self.ROOT_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

    # ============================================================
    # ЗАГРУЗКА ТЕКСТУР (АВТОМАТИЧЕСКАЯ)
    # ============================================================

    def _get_texture(self, symbol):
        """Возвращает текстуру из кэша (для стен, предметов, NPC)"""
        if symbol in self.texture_cache:
            return self.texture_cache[symbol]

        texture_path = None

        # 1. Ищем в SYMBOLS_CONFIG
        config = SYMBOLS_CONFIG.get(symbol, {})
        texture_path = config.get('texture') or config.get('sprite')

        # 2. Если не нашли — ищем в NPC_CONFIG
        if not texture_path:
            npc_config = NPC_CONFIG.get(symbol, {})
            name = npc_config.get('name')
            if name:
                variants = [
                    f"{name}_move_front_1.png",
```

```
                f"{name}_idle_front.png",
                f"{name}_front.png",
            ]
            for variant in variants:
                test_path = os.path.join(self.ROOT_DIR, 'resources', 'npc', name, variant)
                if os.path.exists(test_path):
                    texture_path = test_path
                    break

            if not texture_path:
                folder_path = os.path.join(self.ROOT_DIR, 'resources', 'npc', name)
                if os.path.exists(folder_path):
                    for file in os.listdir(folder_path):
                        if file.endswith('.png'):
                            texture_path = os.path.join(folder_path, file)
                            break

    # 3. Загружаем текстуру
    if texture_path and os.path.exists(texture_path):
        try:
            tex = pygame.image.load(texture_path).convert_alpha()
            self.texture_cache[symbol] = tex
            return tex
        except Exception as e:
            print(f"[Canvas] Ошибка загрузки {texture_path}: {e}")

    self.texture_cache[symbol] = None
    return None

def _get_font(self, size):
    """Возвращает шрифт из кэша"""
    if size not in self.font_cache:
        self.font_cache[size] = pygame.font.Font(None, size)
    return self.font_cache[size]

# ============================================================
# УПРАВЛЕНИЕ КАРТОЙ
# ============================================================

def set_grid(self, grid):
    """Устанавливает карту для отрисовки"""
    self.grid = grid
    self.height = len(grid)
    self.width = len(grid[0]) if grid else 0
    self._center_view()

def _center_view(self):
    if self.width == 0 or self.height == 0:
        return

    max_cell_w = self.rect.width // self.width
    max_cell_h = self.rect.height // self.height
    self.cell_size = min(max_cell_w, max_cell_h, 60)
    self.cell_size = max(self.cell_size, 8)

    total_w = self.width * self.cell_size
    total_h = self.height * self.cell_size
    self.scroll_x = (self.rect.width - total_w) // 2
    self.scroll_y = (self.rect.height - total_h) // 2
```

```
def _clamp_scroll(self):
    if self.width == 0 or self.height == 0:
        return

    total_w = self.width * self.cell_size
    total_h = self.height * self.cell_size

    min_scroll_x = self.rect.width - total_w
    min_scroll_y = self.rect.height - total_h
    max_scroll_x = max(100, self.rect.width // 2)
    max_scroll_y = max(100, self.rect.height // 2)

    if total_w < self.rect.width:
        self.scroll_x = (self.rect.width - total_w) // 2
    else:
        self.scroll_x = max(min_scroll_x - 50, min(self.scroll_x, max_scroll_x))
        self.scroll_x = min(max_scroll_x, self.scroll_x)

    if total_h < self.rect.height:
        self.scroll_y = (self.rect.height - total_h) // 2
    else:
        self.scroll_y = max(min_scroll_y - 50, min(self.scroll_y, max_scroll_y))
        self.scroll_y = min(max_scroll_y, self.scroll_y)

# ============================================================
# ЗУМ
# ============================================================

def zoom_in(self):
    old_size = self.cell_size
    self.cell_size = min(self.cell_size + 2, 80)
    if self.cell_size != old_size:
        self._adjust_scroll_after_zoom(old_size)
        self._clamp_scroll()

def zoom_out(self):
    old_size = self.cell_size
    self.cell_size = max(self.cell_size - 2, 6)
    if self.cell_size != old_size:
        self._adjust_scroll_after_zoom(old_size)
        self._clamp_scroll()

def _adjust_scroll_after_zoom(self, old_size):
    ratio = self.cell_size / old_size
    self.scroll_x = self.scroll_x * ratio
    self.scroll_y = self.scroll_y * ratio

# ============================================================
# РАБОТА С МЫШЬЮ
# ============================================================

def get_cell_at(self, mouse_x, mouse_y):
    if not self.rect.collidepoint(mouse_x, mouse_y):
        return None

    rel_x = mouse_x - self.rect.x - self.scroll_x
    rel_y = mouse_y - self.rect.y - self.scroll_y
```

```
    if rel_x < 0 or rel_y < 0:
        return None

    grid_x = rel_x // self.cell_size
    grid_y = rel_y // self.cell_size

    if 0 <= grid_x < self.width and 0 <= grid_y < self.height:
        return (int(grid_x), int(grid_y))
    return None

def get_cell_rect(self, x, y):
    return pygame.Rect(
        self.rect.x + self.scroll_x + x * self.cell_size,
        self.rect.y + self.scroll_y + y * self.cell_size,
        self.cell_size,
        self.cell_size
    )

def start_drag(self, mouse_x, mouse_y):
    self.dragging = True
    self.drag_start_x = mouse_x
    self.drag_start_y = mouse_y
    self.scroll_start_x = self.scroll_x
    self.scroll_start_y = self.scroll_y

def update_drag(self, mouse_x, mouse_y):
    if self.dragging:
        dx = mouse_x - self.drag_start_x
        dy = mouse_y - self.drag_start_y
        self.scroll_x = self.scroll_start_x + dx
        self.scroll_y = self.scroll_start_y + dy
        self._clamp_scroll()

def end_drag(self):
    self.dragging = False

def update_hover(self, mouse_x, mouse_y):
    if not self.dragging:
        self.hover_cell = self.get_cell_at(mouse_x, mouse_y)

# ============================================================
# ОТРИСОВКА
# ============================================================

def draw(self, screen):
    """Отрисовывает карту с текстурами"""
    pygame.draw.rect(screen, COLORS['background'], self.rect)

    if not self.grid or self.width == 0:
        font = pygame.font.Font(None, 24)
        text = font.render("Загрузите уровень", True, COLORS['text_dim'])
        text_rect = text.get_rect(center=self.rect.center)
        screen.blit(text, text_rect)
        return

    # Видимая область
    start_x = max(0, int(-self.scroll_x // self.cell_size) - 1)
    start_y = max(0, int(-self.scroll_y // self.cell_size) - 1)
    end_x = min(self.width, int((self.rect.width - self.scroll_x) // self.cell_size) + 2)
```

```
    end_y = min(self.height, int((self.rect.height - self.scroll_y) // self.cell_size) + 2)

    font_size = max(8, int(self.cell_size * 0.4))

    for y in range(start_y, end_y):
        for x in range(start_x, end_x):
            symbol = self.grid[y][x]
            rect = self.get_cell_rect(x, y)

            if rect.right < self.rect.x or rect.left > self.rect.right:
                continue
            if rect.bottom < self.rect.y or rect.top > self.rect.bottom:
                continue

            # 1. Текстура (автоматически для всего)
            texture = self._get_texture(symbol)

            if texture:
                scaled = pygame.transform.scale(texture, (rect.width, rect.height))
                screen.blit(scaled, rect)
            else:
                # 2. Цвет (fallback)
                color = SYMBOL_COLORS.get(symbol, (50, 50, 55))
                pygame.draw.rect(screen, color, rect)

                # 3. Текст (если есть что показать)
                if symbol != '_' and symbol != ' ':
                    display_text = symbol[:7] + '…' if len(symbol) > 8 else symbol
                    font = self._get_font(font_size)
                    text = font.render(display_text, True, (255, 255, 255))
                    text_rect = text.get_rect(center=rect.center)
                    screen.blit(text, text_rect)

    # Сетка
    self._draw_grid(screen, start_x, start_y, end_x, end_y)

    # Подсветка клетки под курсором
    if self.hover_cell and not self.dragging:
        x, y = self.hover_cell
        if start_x <= x < end_x and start_y <= y < end_y:
            rect = self.get_cell_rect(x, y)
            pygame.draw.rect(screen, COLORS['selection'], rect, 2)

def _draw_grid(self, screen, start_x, start_y, end_x, end_y):
    for x in range(start_x, end_x + 1):
        pos = self.rect.x + self.scroll_x + x * self.cell_size
        if self.rect.x <= pos <= self.rect.right:
            pygame.draw.line(screen, COLORS['grid'], (pos, self.rect.y), (pos, self.rect.bottom), 1)

    for y in range(start_y, end_y + 1):
        pos = self.rect.y + self.scroll_y + y * self.cell_size
        if self.rect.y <= pos <= self.rect.bottom:
            pygame.draw.line(screen, COLORS['grid'], (self.rect.x, pos), (self.rect.right, pos), 1)
```

=== FILE: map_editor/ui/info_panel.py ===

```
"""Панель информации о клетке — полностью автоматическая"""

import pygame
from ..config import COLORS
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG, WEAPON_CONFIG


class InfoPanel:
    """Отображает информацию о текущей клетке"""

    def __init__(self, rect):
        self.rect = rect
        self.cell_x = None
        self.cell_y = None
        self.symbol = None
        self.tool = 'BRUSH'
        self.selected_symbol = 'M'
        self.has_changes = False
        self.font = pygame.font.Font(None, 16)

    def update(self, cell_pos, symbol):
        if cell_pos:
            self.cell_x, self.cell_y = cell_pos
            self.symbol = symbol
        else:
            self.cell_x = None
            self.cell_y = None
            self.symbol = None

    def update_status(self, tool, selected_symbol, has_changes):
        self.tool = tool
        self.selected_symbol = selected_symbol
        self.has_changes = has_changes

    def draw(self, screen):
        pygame.draw.rect(screen, COLORS['info_bg'], self.rect)

        left_text = ""
        if self.cell_x is not None:
            type_name = self._get_type_name(self.symbol)
            left_text = f"({self.cell_x}, {self.cell_y})  Символ: '{self.symbol}'  Тип: {type_name}"
        else:
            left_text = "Наведите на клетку"

        text_left = self.font.render(left_text, True, COLORS['text'])
        screen.blit(text_left, (self.rect.x + 12, self.rect.y + 10))

        right_text = f"Инструмент: {self.tool.upper()}  |  Объект: '{self.selected_symbol}'"
        if self.has_changes:
            right_text += "  |  * (изменено)"

        text_right = self.font.render(right_text, True, COLORS['text_dim'])
        right_x = self.rect.right - text_right.get_width() - 12
        screen.blit(text_right, (right_x, self.rect.y + 10))
```

```
def _get_type_name(self, symbol):
    """Полностью автоматическое определение типа объекта"""
    if not symbol:
        return "—"

    # ============================================================
    # 1. ПРОВЕРЯЕМ SYMBOLS_CONFIG
    # ============================================================
    config = SYMBOLS_CONFIG.get(symbol, {})
    symbol_type = config.get('type', '')

    if symbol_type == 'wall':
        return "Стена"

    elif symbol_type == 'door':
        door_type = config.get('door_type', 'normal')
        required_key = config.get('required_key')

        if door_type == 'secret':
            return "Секретная дверь"
        elif required_key:
            return f"Дверь (ключ: {required_key})"
        else:
            return "Дверь"

    elif symbol_type == 'exit':
        return "Выход"

    elif symbol_type == 'player_spawn':
        return "Старт"

    elif symbol_type == 'item':
        item_type = config.get('item_type', '')
        weapon_name = config.get('weapon_name')
        key_color = config.get('key_color')

        if item_type == 'health':
            return f"Аптечка (+{config.get('amount', 25)})"
        elif item_type == 'armor':
            return f"Броня (+{config.get('amount', 25)})"
        elif item_type == 'key' and key_color:
            return f"Ключ ({key_color})"
        elif weapon_name:
            return f"Оружие: {weapon_name} (+{config.get('ammo', 0)})"
        else:
            return "Предмет"

    # ============================================================
    # 2. ПРОВЕРЯЕМ NPC_CONFIG
    # ============================================================
    npc_config = NPC_CONFIG.get(symbol, {})
    if npc_config:
        name = npc_config.get('name', 'NPC')
        class_name = npc_config.get('class_name', '')
        if class_name == 'Boss':
            return f"Босс: {name}"
        else:
            return f"NPC: {name}"
```

```
# ============================================================
# 3. ПРОВЕРЯЕМ WEAPON_CONFIG (если оружие не в SYMBOLS_CONFIG)
# ============================================================
weapon_config = WEAPON_CONFIG.get(symbol, {})
if weapon_config:
    return f"Оружие: {symbol}"

# ============================================================
# 4. НЕИЗВЕСТНЫЙ ТИП
# ============================================================
return "Объект"
```

=== FILE: map_editor/ui/menu.py ===

=== FILE: map_editor/ui/toolbar.py ===

```
"""Панель объектов — полностью автоматическая из конфигов"""

import os
import sys
import pygame
from ..config import COLORS, SYMBOL_COLORS, TEXTURES_DIR, NPC_DIR
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
class Toolbar:
    def __init__(self, rect):
        self.rect = rect
        self.items = []
        self.selected_index = 0
        self.scroll_offset = 0
        self.item_height = 48
        self.top_padding = 30

        self._build_items()

    def _build_items(self):
        """Автоматически собирает все объекты из конфигов"""
        self.items = []

        # ============================================================
        # 1. СТЕНЫ
        # ============================================================
        wall_symbols = []
        for symbol, config in SYMBOLS_CONFIG.items():
            if config.get('type') == 'wall':
                wall_symbols.append(symbol)
        wall_symbols.sort()

        self.items.append({'type': 'separator', 'label': 'СТЕНЫ'})
        for symbol in wall_symbols:
            surf = self._load_texture_or_color(symbol)
            self.items.append({
                'type': 'item',
                'symbol': symbol,
                'surface': surf,
                'label': symbol,
                'sub': 'стена'
            })

        # ============================================================
        # 2. ДВЕРИ
        # ============================================================
        door_symbols = []
        for symbol, config in SYMBOLS_CONFIG.items():
            if config.get('type') == 'door':
                door_symbols.append(symbol)

        if door_symbols:
            self.items.append({'type': 'separator', 'label': 'ДВЕРИ'})
            for symbol in door_symbols:
```

```
        surf = self._load_texture_or_color(symbol)
        door_type = SYMBOLS_CONFIG[symbol].get('door_type', 'normal')
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': f'дверь {door_type}'
        })

# ============================================================
# 3. ОБЪЕКТЫ (спавн, выход, пол)
# ============================================================
self.items.append({'type': 'separator', 'label': 'ОБЪЕКТЫ'})

for symbol, config in SYMBOLS_CONFIG.items():
    if config.get('type') == 'player_spawn':
        surf = self._create_surface(symbol, (120, 100, 0))
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': 'спавн'
        })
    elif config.get('type') == 'exit':
        surf = self._create_surface(symbol, (0, 100, 0))
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': 'выход'
        })

surf = self._create_surface('_', (20, 20, 25))
self.items.append({
    'type': 'item',
    'symbol': '_',
    'surface': surf,
    'label': '_',
    'sub': 'пустота'
})

# ============================================================
# 4. ПРЕДМЕТЫ
# ============================================================
item_symbols = []
for symbol, config in SYMBOLS_CONFIG.items():
    if config.get('type') == 'item':
        item_symbols.append(symbol)

if item_symbols:
    self.items.append({'type': 'separator', 'label': 'ПРЕДМЕТЫ'})
    for symbol in item_symbols:
        surf = self._load_texture_or_color(symbol)
        config = SYMBOLS_CONFIG[symbol]
        item_type = config.get('item_type', '')
```

```
            if item_type == 'health':
                sub = 'аптечка +25 HP'
            elif item_type == 'armor':
                sub = 'броня +25 Armor'
            elif config.get('weapon_name'):
                sub = f'{config.get("weapon_name")} (+{config.get("ammo", 0)})'
            else:
                sub = 'предмет'

            self.items.append({
                'type': 'item',
                'symbol': symbol,
                'surface': surf,
                'label': symbol,
                'sub': sub
            })

    # ============================================================
    # 5. NPC
    # ============================================================
    self.items.append({'type': 'separator', 'label': 'NPC'})

    for symbol, config in NPC_CONFIG.items():
        name = config.get('name', 'unknown')
        surf = self._load_npc_sprite(name)
        if surf is None:
            surf = self._create_surface(symbol, (100, 0, 0))

        is_boss = config.get('class_name') == 'Boss'
        self.items.append({
            'type': 'item',
            'symbol': symbol,
            'surface': surf,
            'label': symbol,
            'sub': f'{name} {"(босс)" if is_boss else ""}'
        })

def _load_texture_or_color(self, symbol):
    """Загружает текстуру или создаёт цветной квадрат"""
    config = SYMBOLS_CONFIG.get(symbol, {})
    texture_path = config.get('texture') or config.get('sprite')

    if texture_path:
        full_path = os.path.join(ROOT_DIR, texture_path)
        if os.path.exists(full_path):
            try:
                surf = pygame.image.load(full_path).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))
            except:
                pass

    # Fallback: цвет
    color = SYMBOL_COLORS.get(symbol, (60, 60, 70))
    return self._create_surface(symbol, color)

def _load_npc_sprite(self, name):
    """Загружает спрайт NPC"""
    try:
```

```
            # Новая система: name_move_front_1.png
            path = os.path.join(NPC_DIR, name, f"{name}_move_front_1.png")
            if os.path.exists(path):
                surf = pygame.image.load(path).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))

            # Старая система: name_idle_front.png
            path_old = os.path.join(NPC_DIR, name, f"{name}_idle_front.png")
            if os.path.exists(path_old):
                surf = pygame.image.load(path_old).convert_alpha()
                size = 34
                return pygame.transform.scale(surf, (size, size))
        except:
            pass
        return None

    def _create_surface(self, symbol, color):
        size = 34
        surf = pygame.Surface((size, size))
        surf.fill(color)
        pygame.draw.rect(surf, (80, 80, 90), surf.get_rect(), 1)
        font = pygame.font.Font(None, 22)
        text = font.render(symbol, True, (255, 255, 255))
        text_rect = text.get_rect(center=surf.get_rect().center)
        surf.blit(text, text_rect)
        return surf

    def get_selected_symbol(self):
        if self.selected_index < len(self.items):
            item = self.items[self.selected_index]
            if item['type'] == 'item':
                return item['symbol']
        return 'M'

    def handle_click(self, mouse_x, mouse_y):
        if not self.rect.collidepoint(mouse_x, mouse_y):
            return False

        rel_y = mouse_y - self.rect.y - self.top_padding + self.scroll_offset
        index = rel_y // self.item_height

        if 0 <= index < len(self.items):
            self.selected_index = index
            return True

        return False

    def scroll(self, delta):
        total_height = len(self.items) * self.item_height + self.top_padding + 5
        max_scroll = max(0, total_height - self.rect.height + 10)
        self.scroll_offset = max(0, min(max_scroll, self.scroll_offset + delta))

    def draw(self, screen):
        pygame.draw.rect(screen, COLORS['panel_bg'], self.rect)
        pygame.draw.rect(screen, COLORS['panel_border'], self.rect, 1)

        # Заголовок
        font_title = pygame.font.Font(None, 16)
```

```
title = font_title.render("ОБЪЕКТЫ", True, (240, 240, 240))
screen.blit(title, (self.rect.x + 10, self.rect.y + 5))

# Счётчик
item_count = sum(1 for i in self.items if i['type'] == 'item')
font_count = pygame.font.Font(None, 11)
count_text = font_count.render(f"{item_count} объектов", True, (140, 140, 140))
screen.blit(count_text, (self.rect.x + 10, self.rect.y + 22))

# Список
y = self.rect.y + self.top_padding - self.scroll_offset
font_sep = pygame.font.Font(None, 13)
font_label = pygame.font.Font(None, 16)
font_sub = pygame.font.Font(None, 12)

for i, item in enumerate(self.items):
    if y + self.item_height < self.rect.y or y > self.rect.bottom:
        y += self.item_height
        continue

    rect = pygame.Rect(self.rect.x + 5, y, self.rect.width - 10, self.item_height)

    if item['type'] == 'separator':
        pygame.draw.rect(screen, (50, 50, 60), rect)
        text = font_sep.render(item['label'], True, (200, 200, 220))
        screen.blit(text, (rect.x + 10, rect.y + 14))
    else:
        if i == self.selected_index:
            pygame.draw.rect(screen, (80, 80, 160), rect)
        else:
            pygame.draw.rect(screen, (45, 45, 50), rect)

        if item['surface']:
            icon_rect = item['surface'].get_rect(topleft=(rect.x + 6, rect.y + 8))
            screen.blit(item['surface'], icon_rect)

        label_x = rect.x + 44
        label_y = rect.y + 6

        text = font_label.render(item['label'], True, (255, 255, 255))
        screen.blit(text, (label_x, label_y))

        sub_text = font_sub.render(item['sub'], True, (160, 160, 180))
        screen.blit(sub_text, (label_x, label_y + 22))

        pygame.draw.rect(screen, (70, 70, 80), rect, 1)

    y += self.item_height

# Полоса прокрутки
total_height = len(self.items) * self.item_height + self.top_padding + 5
if total_height > self.rect.height:
    bar_height = max(20, int(self.rect.height * (self.rect.height / total_height)))
    scroll_ratio = self.scroll_offset / (total_height - self.rect.height)
    bar_y = self.rect.y + int((self.rect.height - bar_height) * scroll_ratio)
    pygame.draw.rect(screen, (120, 120, 140), (self.rect.right - 10, bar_y, 6, bar_height))
```

=== FILE: map_editor/ui/tools_panel.py ===

```
"""Панель инструментов"""

import pygame


class ToolsPanel:
    """Панель с инструментами (сверху)"""

    def __init__(self, rect):
        self.rect = rect
        self.tools = []
        self.selected_index = 0
        self.tool_width = 80
        self.tool_height = rect.height - 8
        self.tool_start_x = 130  # отступ слева под заголовок

        self._build_tools()

    def _build_tools(self):
        """Собирает инструменты"""
        self.tools = [
            {'id': 'brush', 'label': 'Кисть', 'icon': ' '},
            {'id': 'eraser', 'label': 'Ластик', 'icon': ' '},
            {'id': 'select', 'label': 'Выделение', 'icon': ' '},
            {'id': 'fill', 'label': 'Заливка', 'icon': ' '},
        ]

    def get_selected_tool(self):
        """Возвращает ID выбранного инструмента"""
        if 0 <= self.selected_index < len(self.tools):
            return self.tools[self.selected_index]['id']
        return 'brush'

    def handle_click(self, mouse_x, mouse_y):
        """Обрабатывает клик по панели"""
        if not self.rect.collidepoint(mouse_x, mouse_y):
            return False

        # Относительная позиция от начала кнопок
        rel_x = mouse_x - self.rect.x - self.tool_start_x

        # Проверяем, что клик в области кнопок
        if rel_x < 0:
            return False

        index = rel_x // (self.tool_width + 4)  # +4 на отступ между кнопками

        if 0 <= index < len(self.tools):
            self.selected_index = index
            print(f"[Инструмент] {self.tools[index]['label']}")
            return True

        return False

    def draw(self, screen):
```

```
"""Отрисовывает панель"""
# Фон
pygame.draw.rect(screen, (45, 45, 50), self.rect)
pygame.draw.rect(screen, (80, 80, 90), self.rect, 1)

# Заголовок слева
font_title = pygame.font.Font(None, 18)
title = font_title.render("Инструменты:", True, (200, 200, 200))
screen.blit(title, (self.rect.x + 8, self.rect.y + 14))

# Кнопки
x = self.rect.x + self.tool_start_x
font_icon = pygame.font.Font(None, 24)
font_label = pygame.font.Font(None, 13)

for i, tool in enumerate(self.tools):
    rect = pygame.Rect(x, self.rect.y + 4, self.tool_width, self.tool_height)

    # Фон кнопки
    if i == self.selected_index:
        pygame.draw.rect(screen, (80, 80, 160), rect)
    else:
        pygame.draw.rect(screen, (50, 50, 55), rect)
    pygame.draw.rect(screen, (70, 70, 80), rect, 1)

    # Иконка
    icon_text = font_icon.render(tool['icon'], True, (255, 255, 255))
    screen.blit(icon_text, (rect.x + rect.width//2 - icon_text.get_width()//2, rect.y + 4))

    # Название
    label_text = font_label.render(tool['label'], True, (200, 200, 200))
    screen.blit(label_text, (rect.x + rect.width//2 - label_text.get_width()//2, rect.y + 30))

    x += self.tool_width + 4
```

=== FILE: map_editor/ui/__init__.py ===

```
"""UI компоненты редактора"""

from .canvas import Canvas
from .info_panel import InfoPanel
```